

T2: GitHub

Introduction

Welcome to the second tutorial for SOFT2412/COMP9412. Today you will be working on GitHub, and exploring its features. Learning this is very essential for your assignment, and to understand the aspects including repository and organisation administration, along with their configurations.

We will cover 9 topics this week:

- Collaborating on GitHub
- Join the University's GitHub Enterprise
- The GitHub Workflow
- GitHub and the command line
- Models for Collaborative Development
- Managing a GitHub Organisation
- Project Administration
- Creating a Project Board
- Exploring features of a GitHub Organisation
- Merge Conflicts
- Bonus - Git Rebase

Collaborating on GitHub

Last week we discussed Git and its basic commands. GitHub is a code hosting platform for version control and collaboration. It lets you and others work together on projects from anywhere. In this tutorial, you are going to learn about GitHub. The University's Code Repository is a software repository management platform using an enterprise edition of GitHub. The Code Repository is a tool to provide a university-wide resource for code management, code review, and general collaboration at The University of Sydney.

In this tutorial, you will work on a group of four people and do the tasks on GitHub.

Join the University's GitHub Enterprise

Before we jump into this week's tutorial, your tutor will add you to the SOFT2412 Organisation under the University's Code Repository. You will be working on your project repositories for this course under this GitHub Organisation. You will also have access to extra features about GitHub like Teams etc. and hence this is also an opportunity for you to explore more about GitHub.

Head over to [Sydney GitHub Enterprise](#), and use your unikey to log in. Once you have successfully logged in, your tutor will add you to the GitHub Organisation during the tutorial.

The GitHub Workflow

GitHub builds collaboration directly into the development process. Work is organized into repositories, where developers can outline requirements or direction and set expectations for team members. Then, using the GitHub flow, developers simply create a branch to work on updates, commit changes to save them, open a pull request to propose and discuss changes, and merge pull requests once everyone is on the same page.

The GitHub flow

The GitHub flow is a lightweight, branch-based workflow built around core Git commands used by teams around the globe—including ours.

The GitHub flow has six steps, each with distinct benefits when implemented:

- **Create a branch:** Topic branches created from the canonical deployment branch (usually `master`) allow teams to contribute to many parallel efforts. Short-lived topic branches, in particular, keep teams focused and results in quick ships.
- **Add commits:** Snapshots of development efforts within a branch create safe, revertible points in the project's history.
- **Open a pull request:** Pull requests publicize a project's ongoing efforts and set the tone for a transparent development process.
- **Discuss and review code:** Teams participate in code reviews by commenting, testing, and reviewing open pull requests. Code review is at the core of an open and participatory culture.
- **Merge:** Upon clicking merge, GitHub automatically performs the equivalent of a local 'git merge' operation. GitHub also keeps the entire branch development history on the merged pull request.
- **Deploy:** Teams can choose the best release cycles or incorporate continuous integration tools and operate with the assurance that code on the deployment branch has gone through a robust workflow.

GitHub and the Command Line

Below are simple examples. **You are encouraged to read them among groups to understand how you can work in GitHub.** You will be creating your own repositories to work on in the tutorial later on. These are reference commands so **do not put this in your command line.**



You may run the commands outside Ed. The below code will not run on Ed due to limitations of downloading from links. In fact, this will be good practice for you to familiarize yourself with your own laptop's / pc's command line. Ed is only here to give you a quick alternative and help make the tutorials easy and interactive.

Example: Contribute to an existing repository

```
# download a repository on GitHub to our machine
git clone https://github.sydney.edu.au/demo/demo.git

# change into the `demo` directory
cd demo

# create a new branch to store any new changes
git branch my-branch

# switch to that branch (line of development)
git checkout my-branch

# make changes, for example, edit `file1.md` and `file2.md` using the text editor

# stage the changed files
git add file1.md file2.md

# take a snapshot of the staging area (anything that's been added)
git commit -m "my snapshot"

# push changes to github
git push --set-upstream origin my-branch
```

Note that `--set-upstream` is same as `-u`

Example: Start a new repository and publish it to GitHub

First, you will need to create a new repository on GitHub. The next parts of the tutorial will describe how to create a new repository.

```
# create a new directory, and initialize it with git-specific functions
git init my-repo
```

```
# change into the `my-repo` directory
cd my-repo

# create the first file in the project
touch README.md

# git isn't aware of the file, stage it
git add README.md

# take a snapshot of the staging area
git commit -m "add README to initial commit"

# provide the path for the repository you created on github
git remote add origin https://github.sydney.edu.au/SOT2412-2022S2/YOUR-REPOSITORY.git

# push changes to github
git push --set-upstream origin master
```

Example: contribute to an existing branch on GitHub

```
# assumption: a project called `repo` already exists on the machine, and a new branch has been pushed to the repository

# change into the `repo` directory
cd repo

# update all remote tracking branches, and the currently checked out branch
git pull

# change into the existing branch called `feature-a`
git checkout feature-a

# make changes, for example, edit `file1.md` using the text editor

# stage the changed file
git add file1.md

# take a snapshot of the staging area
git commit -m "edit file1"

# push changes to github
git push
```



Models for Collaborative Development

There are two primary ways people collaborate on GitHub:

- Shared repository
- Fork and pull

With a *shared repository*, individuals and teams are explicitly designated as contributors with read, write, or administrator access. This simple permission structure, combined with features like protected branches and Marketplace, helps teams progress quickly when they adopt GitHub.

For an open-source project, or for projects to which anyone can contribute, managing individual permissions can be challenging, but a *fork and pull* model allows anyone who can view the project to contribute. A fork is a copy of a project under a developer's personal account. Every developer has full control of their fork and is free to implement a fix or new feature. Work completed in forks is either kept separate or is surfaced back to the original project via a pull request. There, maintainers can review the suggested changes before they're merged.

Managing a GitHub Organisation

Managing an organisation

In addition to single-user accounts, GitHub has what are called Organisations. Like personal accounts, Organisational accounts have a namespace where all their projects exist, but many other things are different. These accounts represent a group of people with shared ownership of projects, and there are many tools to manage subgroups of those people. Normally these accounts are used for Open Source groups (such as “perl” or “rails”) or within companies.

We have already created an organisation: [SOFT2412-COMP9412-2024S2](#)

Now, you will learn how to maintain repositories, projects and teams in your organisation.

Maintaining a Project

We will look at how to create, maintain and administer our own projects.

Creating a New Repository

Let's create a new repository to share our project code with. Start by clicking the button on the right-hand side of the dashboard. This takes you to the “new repository” form:

Create a new repository

A repository contains all the files for your project, including the revision history.

Owner



SOFT2412-Agile-Software-Development ▾

Repository name

/

Great repository names are short and memorable. Need inspiration? How about **bookish-system**.

Description (optional)

☐  **Public**

Any logged in user can see this repository. You choose who can commit.

☒  **Private**

You choose who can see and commit to this repository.

☐ **Initialize this repository with a README**

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: **None** ▾

Create repository

All you really have to do here is provide a project name; the rest of the fields are completely optional. For now, just click the "Create Repository" button, and boom – you have a new repository on GitHub, named `SOFT2412-2020S2/<project_name>`.

Since you have no code there yet, GitHub will show you instructions for how to create a brand-new Git repository or connect an existing Git project. Do not initialize the repository with a README, .gitignore or License. This empty repository will await your code.

Adding Collaborators

If you're working with other people to whom you want to give commit access, you need to add them as "collaborators". If Ben, Jeff, and Louise all sign up for accounts on GitHub, and you want to give them push access to your repository, you can add them to your project. Doing so will give them "push" access, which means they have both read and write access to the project and Git repository. You can add using the following button from Repository Homepage:

Give access to the people you work with

You should give access to the collaborators and teams you need to work with.

Add teams and collaborators

Just type a username into the box, and click “Add collaborator.” You can repeat this as many times as you like to grant access to everyone you like. If you need to revoke access, just click the “X” on the right-hand side of their row. For now, just add one group member to your repository.

Collaborators

This repository doesn't have any collaborators yet. Use the form below to add a collaborator.

Search by username, full name or email address

You'll only be able to find a GitHub user by their email address if they've chosen to list it publicly. Otherwise, use their username instead.

Add collaborator

Concepts of Fork and Pull

Forking Project

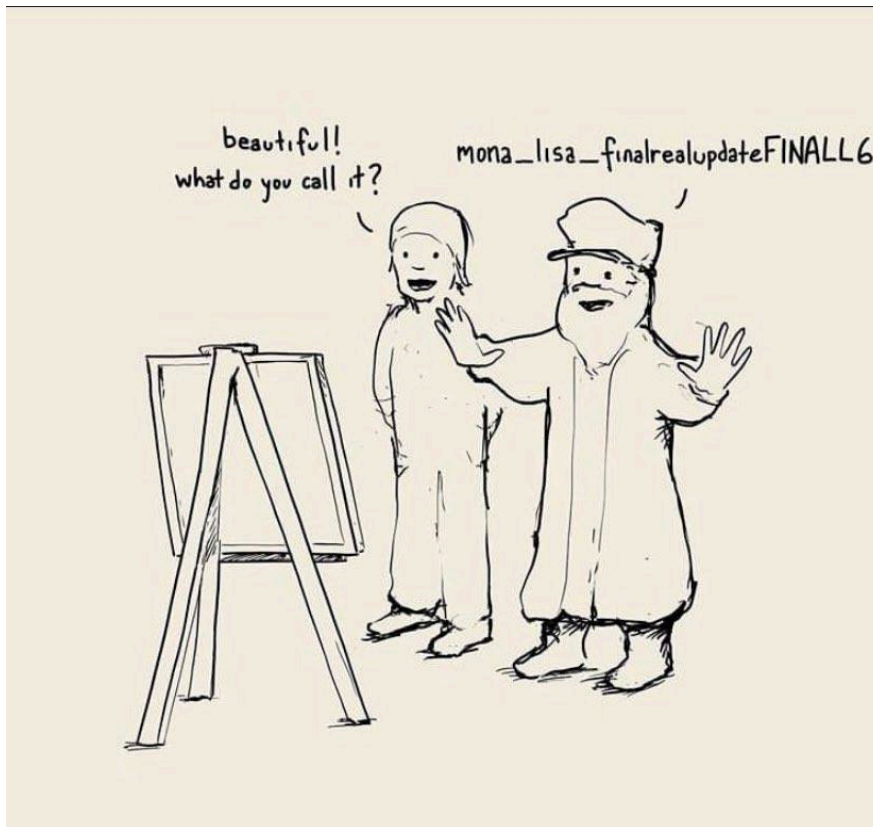
If you want to contribute to an existing project to which you don't have push access, you can “fork” the project. When you “fork” a project, GitHub will make a copy of the project that is entirely yours; it lives in your namespace, and you can push to it. To fork a project, visit the project page and click the “Fork” button at the top-right of the page.



Figure: The “Fork” button. [Credit: Pro Git (2nd Ed.) - Scott Chacon and Ben Straub]

Creating a Pull Request

Create a pull request to propose and collaborate on changes to a repository. These changes are proposed in a *branch*, which ensures that the `master` branch only contains finished and approved work.



Pull requests can only be opened if there are differences between your branch and the upstream branch. You can specify which branch you'd like to merge your changes into when you create your pull request.

If you don't have write access to the repository where you'd like to create a pull request, you must create a fork, or copy, of the repository first.

Here is an example from the Pro Git book on pull requests (The example below is for reference only.



You won't work on this repo since it is outside the enterprise, you may still try it out in your own time).

Tony is looking for code to run on his Arduino programmable microcontroller and has found a great program file on GitHub at <https://github.com/schacon/blink> (Links to an external site.).

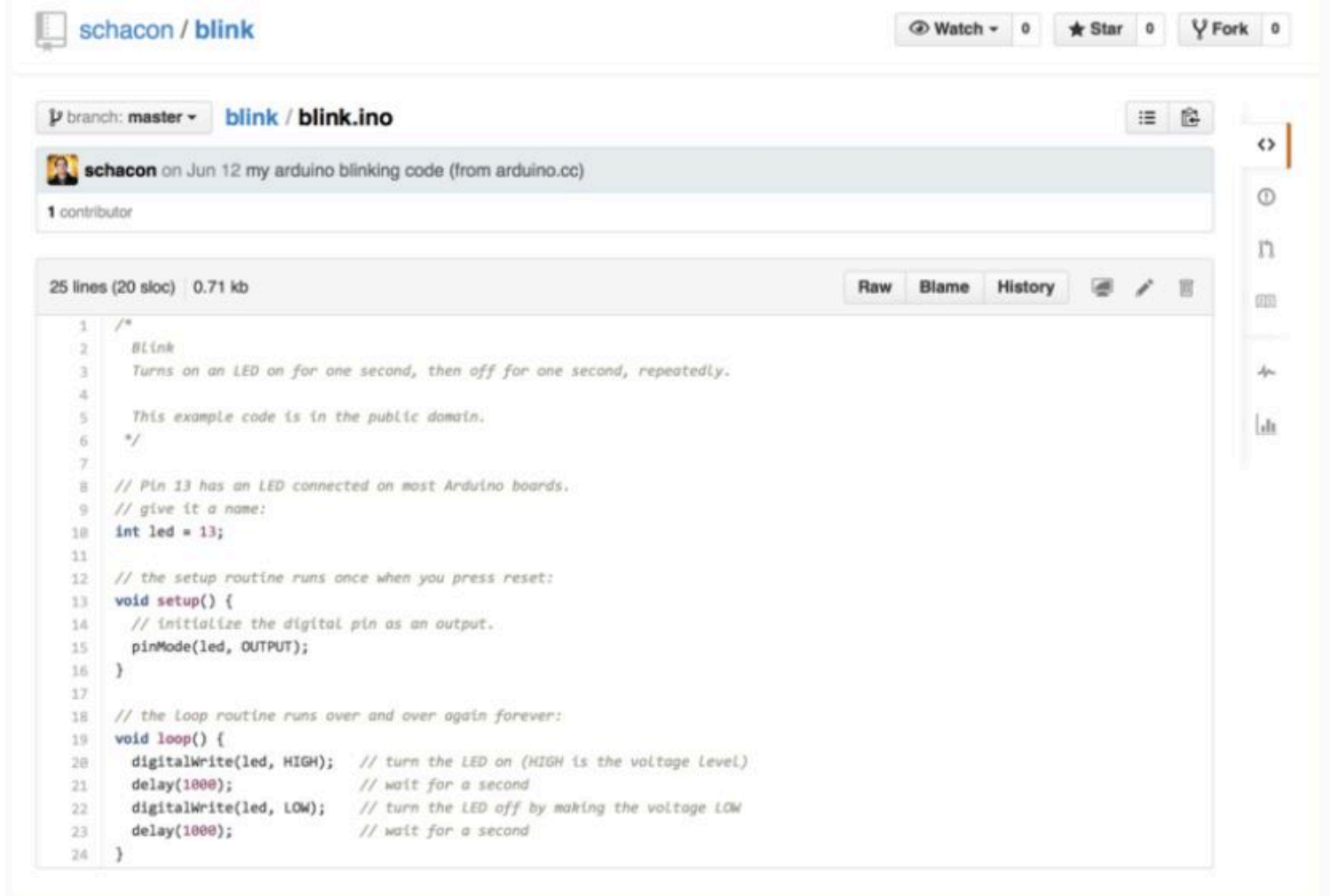


Figure: The project we want to contribute to [Credit [1]]

The only problem is that the blinking rate is too fast. We think it's much nicer to wait 3 seconds instead of 1 in between each state change. So let's improve the program and submit it back to the project as a proposed change.

First, we click the *Fork* button as mentioned earlier to get our own copy of the project. Our user name here is "tonychacon" so our copy of this project is at <https://github.com/tonychacon/blink> and that's where we can edit it. We will clone it locally, create a topic branch, make the code change and finally push that change back up to GitHub.

```
$ git clone https://github.com/tonychacon/blink (1)
Cloning into 'blink'...
```

```
$ cd blink
$ git checkout -b slow-blink (2)
Switched to a new branch 'slow-blink'
```

```
$ sed -i '' 's/1000/3000/' blink.ino (macOS) (3)
# If you're on a Linux system, do this instead:
# $ sed -i 's/1000/3000/' blink.ino (3)
```

```
$ git diff --word-diff (4)
```

```

diff --git a/blink.ino b/blink.ino
index 15b9911..a6cc5a5 100644
--- a/blink.ino
+++ b/blink.ino
@@ -18,7 +18,7 @@ void setup() {
// the loop routine runs over and over again forever:
void loop() {
    digitalWrite(led, HIGH);    // turn the LED on (HIGH is the voltage level)
    [-delay(1000);-][+delay(3000);+]          // wait for a second
    digitalWrite(led, LOW);     // turn the LED off by making the voltage LOW
    [-delay(1000);-][+delay(3000);+]          // wait for a second
}

$ git commit -a -m 'three seconds is better' (5)
[slow-blink 5ca509d] three seconds is better
1 file changed, 2 insertions(+), 2 deletions(-)

$ git push origin slow-blink (6)
Username for 'https://github.com': tonychacon
Password for 'https://tonychacon@github.com':
Counting objects: 5, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 340 bytes | 0 bytes/s, done.
Total 3 (delta 1), reused 0 (delta 0)
To https://github.com/tonychacon/blink
 * [new branch]      slow-blink -> slow-blink

```

- Clone our fork of the project locally
- Create a descriptive topic branch
- Make our change to the code
- Check that the change is good
- Commit our change to the topic branch
- Push our new topic branch back up to our GitHub fork

Now if we go back to our fork on GitHub, we can see that GitHub noticed that we pushed a new topic branch up and presents us with a big green button to check out our changes and open a Pull Request to the original project.

You can alternatively go to the “Branches” page at <https://github.com/<user>/<project>/branches> to locate your branch and open a new Pull Request from there.

Example file to blink the LED on an Arduino — Edit

2 commits

2 branches

0 releases

1 contributor

Your recently pushed branches:

slow-blink (less than a minute ago)

Compare & pull request

branch: master blink / +

This branch is even with schacon:master

Pull Request

Compare

Create README.md

schacon authored on Jun 12

latest commit bbc80f9b29

README.md	Create README.md	4 months ago
blink.ino	my arduino blinking code (from arduino.cc)	4 months ago

README.md

Blink

This repository has an example file to blink the LED on an Arduino board.

Code

Pull Requests 0

Wiki

Pulse

Graphs

Settings

HTTPS clone URL

https://github.com/

You can clone with HTTPS, SSH, or Subversion.

Clone in Desktop

Download ZIP

Figure: Pull Request Button

If we click that green button, we'll see a screen that asks us to give our Pull Request a title and description. It is almost always worthwhile to put some effort into this, since a good description helps the owner of the original project determine what you were trying to do, whether your proposed changes are correct, and whether accepting the changes would improve the original project.

We also see a list of the commits in our topic branch that are "ahead" of the master branch (in this case, just the one) and a unified diff of all the changes that will be made should this branch get merged by the project owner.

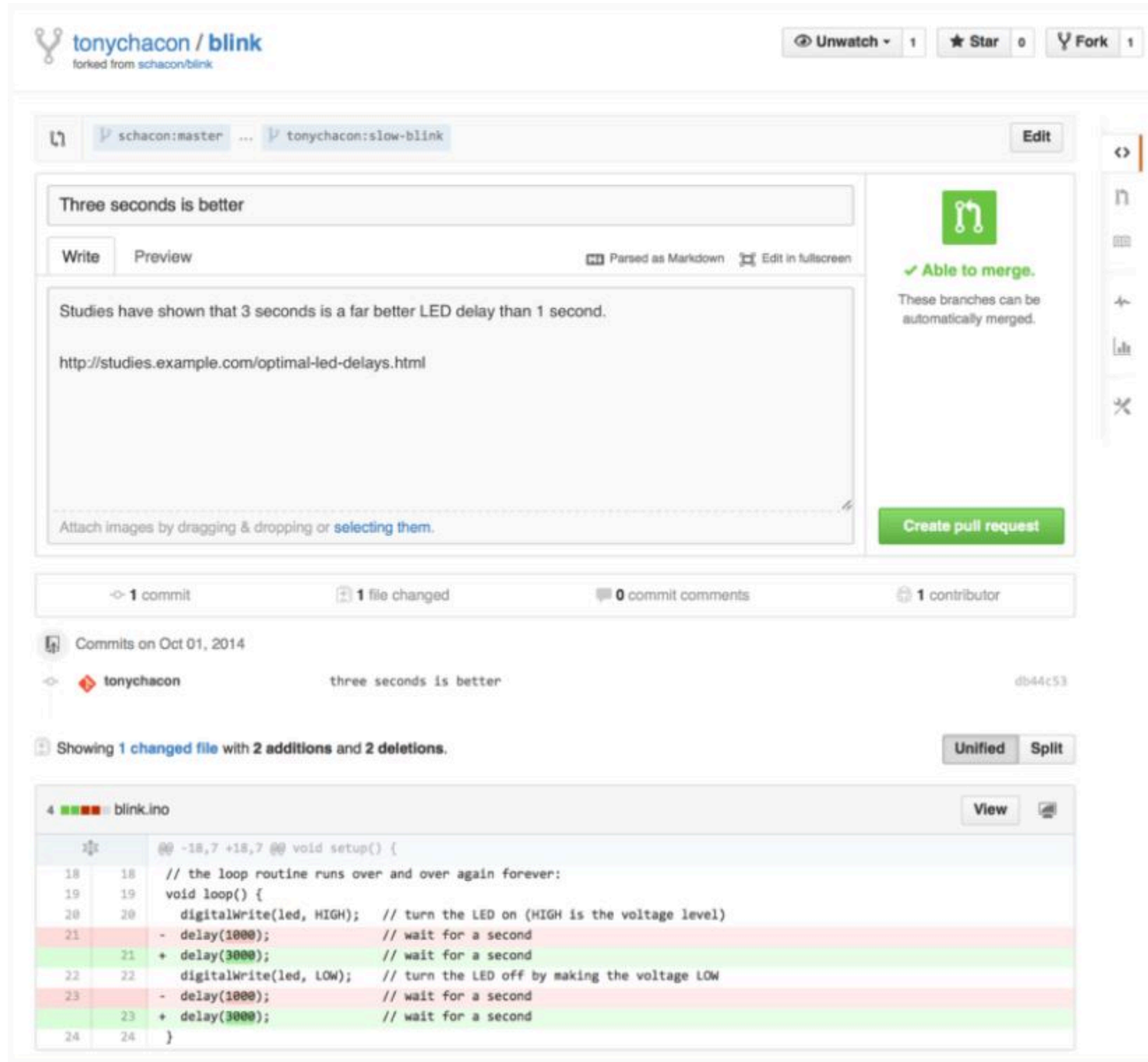


Figure: Pull request Creation Page

Creating a Team

You can create independent or nested teams to manage repository permissions and mentions for groups of people.

Managing team access to an organization repository

Organization owners and team maintainers can add repositories to a team, as well as change the team's read, write, and admin access to the repository.

Special Files

There are a couple of special files that GitHub will notice if they are present in your repository. They

are `README` and `CONTRIBUTING` file. The idea behind the contributing file is that you can specify specific things you want or don't want in a Pull Request sent to your project. This way people may actually read the guidelines before opening the Pull Request.

Project Administration

Generally, there are not a lot of administrative things you can do with a single project, but there are a couple of items that might be of interest.

Changing the Default Branch

If you are using a branch other than “master” as your default branch that you want people to open Pull Requests on or see by default, you can change that in your repository’s settings page under the “Branches” tab.

Default branch

The default branch is considered the “base” branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

gh-pages ▾

Update

Transferring a Project

If you would like to transfer a project to another user or an organization in GitHub, there is a “Transfer ownership” option at the bottom of the “Options” tab of your repository settings page that allows you to do this.

Danger Zone

Make this repository public
Make this repository visible to anyone.

Make public

Transfer ownership
Transfer this repository to another user or to an organization where you have the ability to create repositories.

Transfer

Archive this repository
Mark this repository as archived and read-only.

Archive this repository

Delete this repository
Once you delete a repository, there is no going back. Please be certain.

Delete this repository

Creating a Project Board

Project boards can be used to create customised workflows to suit your needs, like tracking and prioritising specific feature work, comprehensive roadmaps, or even release checklists.

You can use a project board template to create a project board with automation already configured. For more information. Once you've created your project board, you can add issues, pull requests, and notes to it.

You can add issues and pull requests to a project board in the form of cards and triage them into columns.

This is very similar to Trello, a project backlog tool you will be used in one of your assignments and will be learning about in the coming tutorials.

rhianvanesch.com

Updated now

Filter cards

3 To do

ⓘ Add an RSS feed for posts
#3 opened by escherina
enhancement

ⓘ Add posts section
#8 opened by escherina
enhancement

ⓘ Add a 'uses' page
#12 opened by escherina
enhancement

Automated as To do

Manage

0 In progress

Automated as In progress

Manage

3 Done

ⓘ Add syntax highlighting for code blocks
#4 opened by escherina
enhancement

ⓘ Restore full layout
#9 opened by escherina
enhancement

ⓘ Add a favicon
#5 opened by escherina
enhancement

Automated as Done

Manage

Exploring features of a GitHub Organisation

There are some other features that you can explore. This can be done during your tutorial time or at your own time.

Add collaborators

1. Add the provided code to your repository. [Online Survey App.zip](#)
2. Add a collaborator (from your group) into the repository.
3. Verify the access of the collaborator you have added.

Creating a team

1. Create teams for your project.
2. Add the collaborators to different teams.

Managing team access

1. Create a repository.
2. Give your team access to the repository you have created.
3. How have you arranged the access levels of different team members? Why?

Project board

1. Create a repository project board under our organisation.
2. Add issues and pull requests to the project board you have created.

Merge Conflicts

Git can often resolve differences between merged branches. Usually, the changes are on different lines, or even in different files, which makes the merge simple for computers to understand. However, sometimes there are competing changes that Git needs your help with to decide which changes to incorporate in the final merge. Often, merge conflicts happen when people make different changes to the same line of the same file, or when one person edits a file and another person deletes the same file. You must resolve the conflict before you can merge the branches.

If you have a merge conflict between your compare branch and base branch on GitHub, you can view the files with conflicting changes above the **Merge pull request** button, which will be deactivated.

You cannot merge your pull request branch on GitHub until you resolve the merge conflict. To resolve the merge conflict, you must manually select the changes that you want to keep in the final merge on the command line or on GitHub.

You can only resolve the merge conflict on GitHub, if your merge conflict is caused by competing line changes, such as when people make different changes to the same line of the same file on different branches in your Git repository. For all other merge conflicts, you must resolve the merge conflict locally on the command line.

If you have a merge conflict on the command line, you cannot push your local changes to GitHub until you resolve the merge conflict locally on your computer. If you try merging branches on the command line that have a merge conflict, you'll get an error message.

Bonus - Rebase

Last week, we covered certain basic git commands that are essential for using git. Now, we will touch over what git rebase is, given that we have covered GitHub. You will find this helpful in your assignments and all times of git usage.

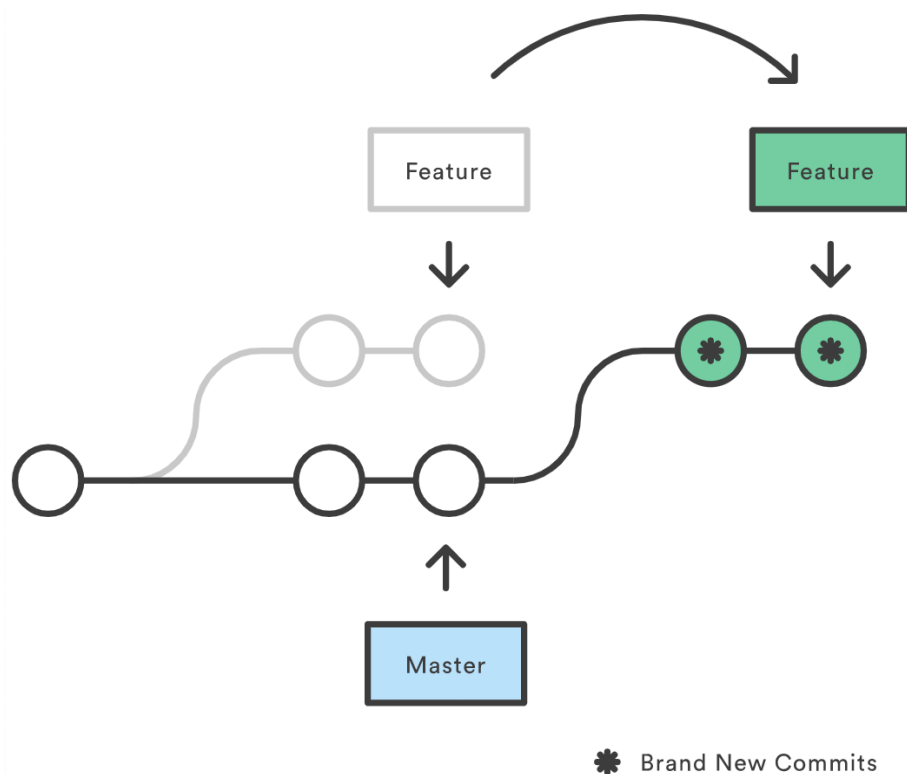
Rebase is one of two Git utilities that specialises **in integrating changes from one branch onto another**. The other change integration utility is `git merge`. Merge is always a forward moving change record. Alternatively, rebase has powerful **history rewriting features** (Atlassian).

Rebasing is the process of moving or combining a sequence of commits to a new base commit.

```
git rebase <base>
```

(Note that <base> can be anything, a branch name, a SHA hash of a commit, `HEAD`, or a tag)

The above command will rebase the current branch into the <base>. This means, that running this command will take the commits of the current working branch and will apply them to the head of the passed branch.



The question arises: Why would anyone use rebase? The answer in this scenario is to maintain a linear history.

Consider a situation where the master branch has progressed since you branched out to another branch to work on some feature. Now you want the latest updates of the master branch on your feature branch. Well at this point, one might think: let's just merge master. But its important to realise

that while this works, it will make the git history messy. Using `git rebase` here keeps your branch's history clean so it appears as if you branched off the latest master branch commit (see image above).

To be able to alter and polish your history, you would need to pass the interactive flag in `git rebase` as shown below:

```
git rebase --interactive <base>  
// or  
git rebase -i <base>
```

This will launch an interactive rebasing session where you can choose how individual commits are to be transferred to the new base. There are several options available that you will see in the editor that will pop up when you run the command.

This way, you have complete control over what your project history will look like.

However, there are some dangers of git rebase that must be kept in mind while using it:

1. Many conflicts can occur while rebasing several commits that have deviated codelines
2. Rebasing rewrites history, so force pushing the rebased results will overwrite any other developers work that he/she might have pushed up already that you haven't pulled
3. Loss of commits can take place here as git rebase allows dropping commits along the path

Setting up your local repository for the assignment

In previous years, some students forgot to set up their assignment project repositories properly, resulting in the partial or even total loss of their assignment marks. To avoid that situation, we kindly ask you to set up your local project repository before the end of the lab. Should you encounter any difficulties, alert your tutor and ask for help.

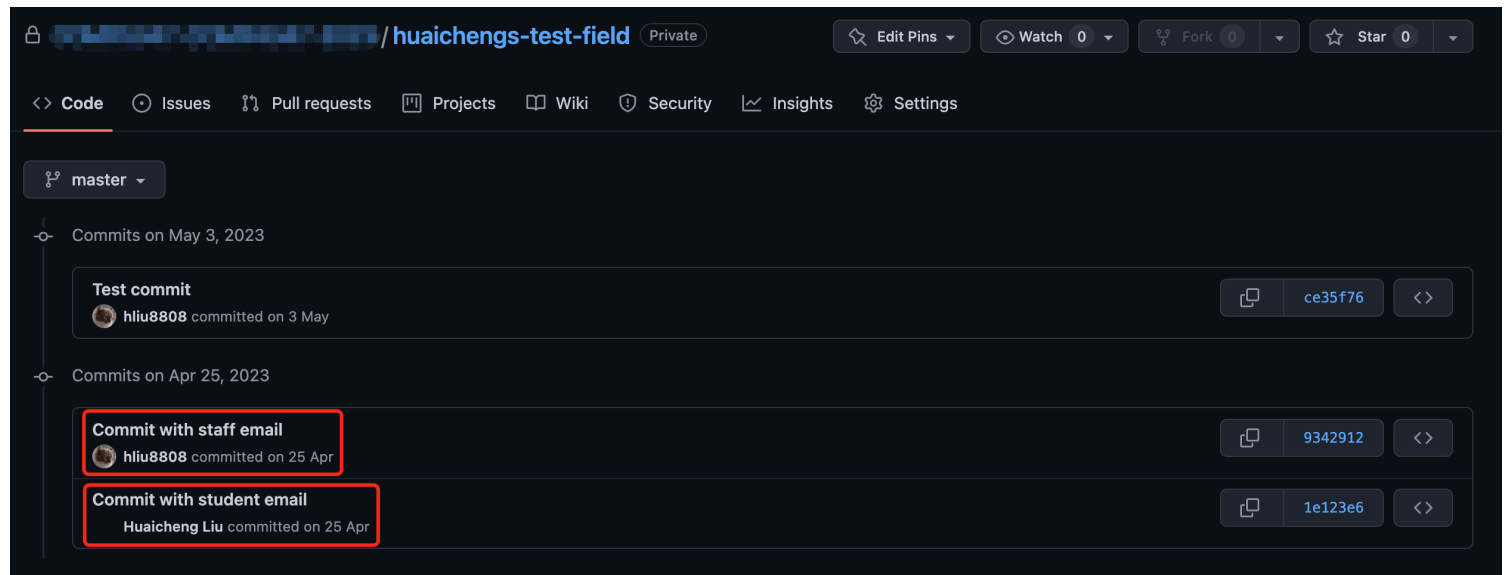
To set up your local repo, first head to the root directory of the project you just cloned

```
cd [path to your project]
```

Then, run these two commands separately. Please make sure you replace the placeholders with your name and **USYD email address**, do **not** use your personal email

```
git config --local user.name "<Your Name>"
git config --local user.email "<Your USYD Email Address>"
```

After you finished with this configuration, it is suggested that you do a test commit and push to your remote repository. Simply make an irrelevant change to the existing files (add a new line etc.), or make an empty dummy file, commit, and push. Then, head to your GitHub Enterprise's repo page and look for the commit history.



The screenshot shows the commit history of a GitHub repository named 'huaichengs-test-field'. The repository is private and has 0 stars and 0 forks. The commit history is filtered by the 'master' branch. There are three commits shown, all dated May 3, 2023. The first commit is titled 'Test commit' and was made by user 'hliu8808'. The second commit is titled 'Commit with staff email' and was made by user 'hliu8808'. The third commit is titled 'Commit with student email' and was made by user 'Huaicheng Liu'. The second and third commits are highlighted with red boxes.

Once you're there, you'll be able to tell if your local configuration is correct if author's name is your unikey, and there's an avatar.



Students with USYD work email

For those who are also working for the university, or otherwise obtained an `fname.lname@sydney.edu.au` email address, please use that instead of the `unikey@uni.sydney.edu.au` one, as it is your new primary email address in our system.